DAPOS database guide

A plain-language map of the PostgreSQL database behind the DAPOS portal.

Source: `backend/prisma/schema.prisma`. Names below are the real table names in the database. This guide does not change the database.

> **How to read a connection.** The parent is the "one" side. The child is the "many" side. The child stores the parent's id in a foreign key. Example: one vendor can supply many products, so each product row keeps `vendorId`. Every real link in this database is one-to-many. There is no one-to-one pair and no many-to-many join table. An order and a product meet through `order_items`.

There are **18 tables** and **34 foreign keys**. Almost every row belongs to one store. There is no separate permissions table. A role's permissions are a text list stored on the role, and one person can have a custom list instead.

1. Store and staff

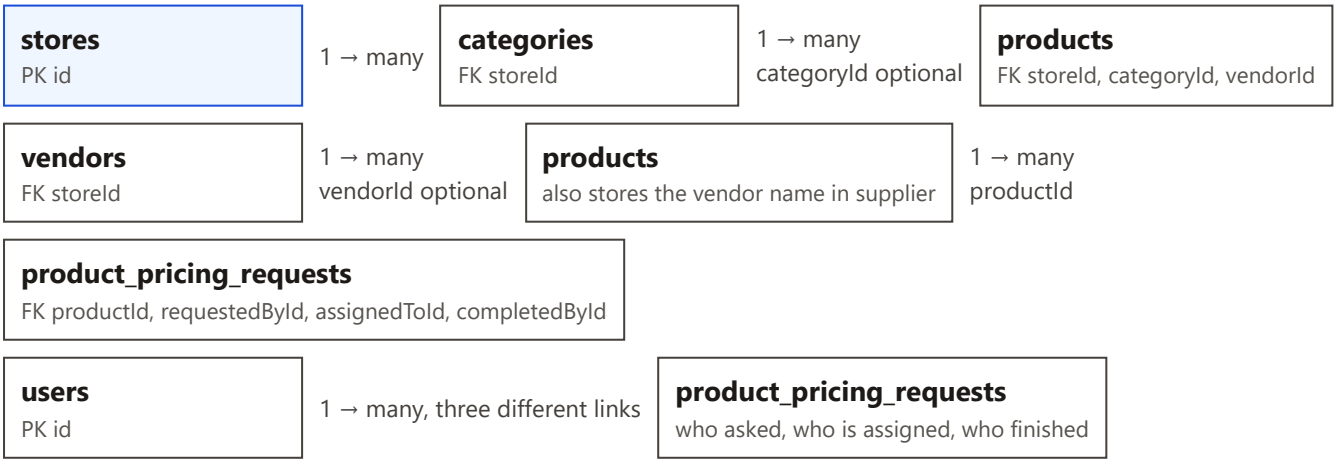


stores is the shop: name, address, phone, currency, tax rate, and whether it is active. `rolePolicies` is an older copy of role permissions stored as JSON, not a link to another table.

store_roles is a job such as Administrator, Cashier, or a custom role. `permissions` is the list of what that job is allowed to do.

users is a staff person who can sign in, not a shop customer. `storeId` says which shop they work in. `storeRoleId` says which job they have, and it can be empty. `customPolicies` is an optional personal permission list.

2. Products, categories, vendors, and pricing



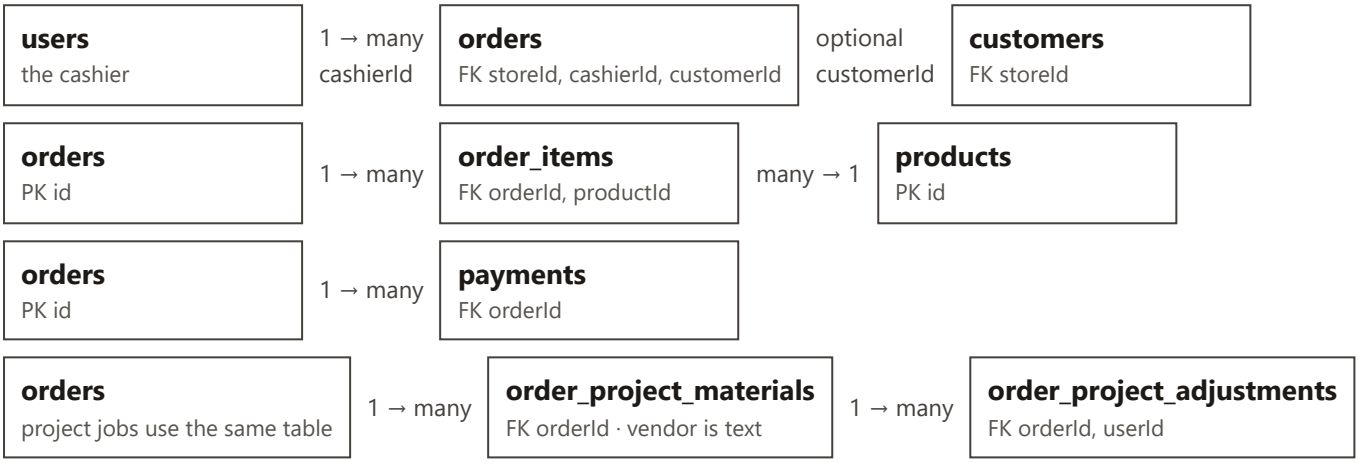
categories groups products, for example Beverages. A product does not have to have a category.

vendors is the supplier list. The unique name check uses `normalizedName` so the same vendor is not stored twice under a slightly different spelling.

products is an item. `vendorId` points at the chosen supplier. `supplier` also keeps that vendor's name as text. `isPriced` is false until a price is published, so the point of sale will not sell it yet. `price` starts at 0.

product_pricing_requests is the request that a new item still needs a price. It records who submitted the details, who is assigned, who completed it, the status (PENDING or COMPLETED), the submission time, and the completion time.

3. Customers and sales



customers is a shopper, used for loyalty and for linking some sales. A sale does not have to have a customer.

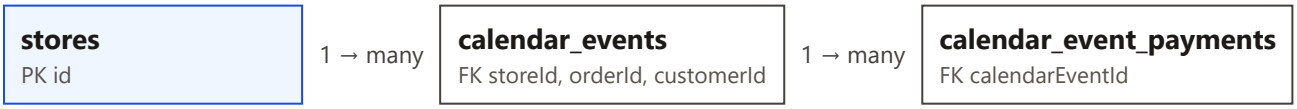
orders is both a regular sale and a project job. `orderType` says which. `cashierId` is the staff user who created it.

order_items is one line on a receipt. It points at the product and also copies the product name, SKU, and the price that was charged, so later price changes do not rewrite old receipts.

payments is money taken for that order. One order can have more than one payment.

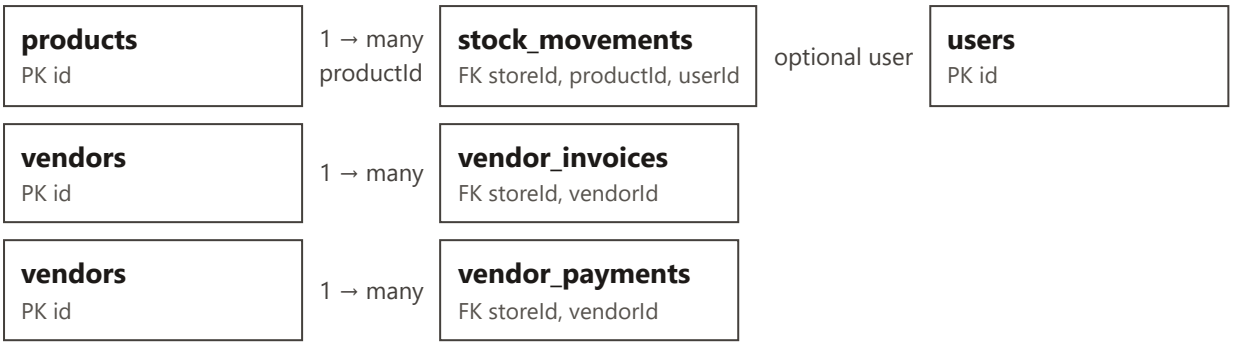
order_project_materials lists raw materials on a project. The vendor on that line is a typed name, not a link to `vendors`. **order_project_adjustments** is a note about a change, and who wrote it.

4. Calendar



An activity can optionally point at one order and one customer. It also copies `customerName` and `customerPhone` as text. Partial payments for an activity are separate rows in `calendar_event_payments`.

5. Stock and the vendor ledger



stock_movements is the history of stock going up or down. `orderId` on that table is stored as text only. The database does not force it to match a real order.

vendor_invoices is a bill from a supplier. **vendor_payments** is money paid to that supplier. Both belong to one vendor.

Full relationship list

Parent (one)	Child (many)	Foreign key on the child	Required?
stores	store_roles	storeId	Yes
stores	users	storeId	Yes
store_roles	users	storeRoleId	No
stores	categories	storeId	Yes
stores	products	storeId	Yes
categories	products	categoryId	No
vendors	products	vendorId	No
stores	product_pricing_requests	storeId	Yes
products	product_pricing_requests	productId	Yes
users	product_pricing_requests	requestedById	Yes
users	product_pricing_requests	assignedToId	No
users	product_pricing_requests	completedById	No
stores	customers	storeId	Yes
stores	orders	storeId	Yes
users	orders	cashierId	Yes
customers	orders	customerId	No
orders	order_items	orderId	Yes
products	order_items	productId	Yes
orders	payments	orderId	Yes
orders	order_project_materials	orderId	Yes
orders	order_project_adjustments	orderId	Yes
users	order_project_adjustments	userId	Yes
stores	calendar_events	storeId	Yes
orders	calendar_events	orderId	No
customers	calendar_events	customerId	No
calendar_events	calendar_event_payments	calendarEventId	Yes
stores	stock_movements	storeId	Yes
products	stock_movements	productId	Yes
users	stock_movements	userId	No

Parent (one)	Child (many)	Foreign key on the child	Required?
stores	vendors	storeId	Yes
stores	vendor_invoices	storeId	Yes
vendors	vendor_invoices	vendorId	Yes
stores	vendor_payments	storeId	Yes
vendors	vendor_payments	vendorId	Yes

What happens when you add a product

1. Item details are saved

A new row is inserted into `products`. `storeId` is your store. If you chose a category, `categoryId` points at `categories`. If you chose a supplier, `vendorId` points at `vendors`, and `supplier` is filled with that vendor's name. `price` is 0 and `isPriced` is false, so it cannot be sold yet.

2. A pricing request is opened

A row is inserted into `product_pricing_requests`. `productId` points at the new product. `requestedById` points at the staff user who submitted the details. `status` is PENDING. `submittedAt` is the submission time. Assignment and completion stay empty.

3. An authorized person sets the price

The product row is updated with `price`, `cost`, and `isPriced` = true. The request becomes COMPLETED. `completedAt` and `completedById` record when and who. The item can then be sold.

4. A later sale

An `orders` row is created for the cashier and, if used, the customer. Each line becomes an `order_items` row pointing at the order and the product, with a copy of the name and the price charged. A `payments` row records the money. A `stock_movements` row records the stock drop.

Columns that look connected but are not enforced

These values are stored as ids or names, but the schema does not declare a foreign key. The database will not stop a bad id, and deleting the related row will not update them.

- `stock_movements.orderId` is not linked to `orders`.
- `vendor_invoices.sourceOrderId` is not linked to `orders`.
- `vendor_invoices.orderProjectMaterialId` is unique text, not a link to `order_project_materials`.
- `vendor_invoices.createdById` and `vendor_payments.createdById` are not linked to `users`.
- `order_project_materials.vendor` is a typed name, not `vendors.id`.

The same fact stored twice

- `products.supplier` copies the vendor name even though `products.vendorId` already points at the vendor. Renaming a vendor can leave the old name on the product.

- Permissions live in three JSON places: `store_roles.permissions`, `users.customPolicies`, and the older `stores.rolePolicies`. There is no permissions table.
- `calendar_events` copies `customerName` and `customerPhone` beside `customerId`.
- `order_items` copies product name, SKU, and price on purpose, so old receipts stay accurate.

Suggestions only — nothing was changed

- Make the text id columns above into real foreign keys.
- Link project materials to `vendors` the same way products are linked.
- Keep the vendor name in one place, or update the copied name when a vendor is renamed.
- Allow only one open pricing request per product.

DAPOS database guide. Generated from the Prisma schema. No tables were invented, and the database structure was not modified.